CURSO · DÍA 3 DE 3

Datos, redes, Compose e infraestructura

De un contenedor aislado a una aplicación completa, y de ahí a vuestros servidores



Jorge López Blasco · Formador IT

Repaso rápido del día 2

1 ¿Por qué dos etapas?

¿Qué hay en la etapa build que no queremos en producción? ¿Qué copia exactamente COPY --from=build?

2 ¿Por qué el .csproj primero?

¿Qué capa se reutiliza cuando cambio una línea de Program.cs?

3 ¿Cómo sobrescribo ConnectionStrings:Default?

¿Qué variable de entorno uso y por qué gana al appsettings.json?

4 ¿Y .NET Framework?

¿Qué tipo de contenedor necesita y cuál es la estrategia recomendada?

Agenda de hoy

09:00	Repaso del día 2
09:10	Datos: volúmenes y montajes
09:30	Redes y comunicación entre contenedores
09:50	**Lab 07 · Volúmenes y redes**
10:50	Descanso
11:10	Docker Compose
11:35	**Lab 08 · Docker Compose**
12:45	Buenas prácticas
13:00	De IIS a contenedores: infraestructura y hoja de ruta
13:35	**Lab 09 · Reto final (partes 1-3) y cierre del curso**

01

Datos

Que los datos sobrevivan al contenedor



Tres formas de montar almacenamiento

V

Volumen con nombre

Almacenamiento gestionado por Docker, independiente del contenedor. Es la opción para datos: bases de datos, ficheros subidos.

-v sqldata:/var/opt/mssql

B

Bind mount

Una carpeta o fichero concreto del equipo, visible dentro del contenedor. Ideal en desarrollo y para ficheros de configuración.

-v "${PWD}\web:/usr/share/nginx/html:ro"

T

tmpfs

Almacenamiento en memoria, que desaparece al parar el contenedor. Para ficheros temporales o sensibles que no deben tocar el disco.

--tmpfs /tmp

SQL Server en contenedor: dónde va cada cosa

Contenedor sql1 (desechable)

- Imagen mssql/server:2022
- Binarios de SQL Server
- Configuración: ACCEPT_EULA, MSSQL_SA_PASSWORD
- /var/opt/mssql → montado

→

Volumen sqldata (persistente)

Ficheros .mdf y .ldf, logs de error, configuración de la instancia

Actualizar SQL Server: nueva imagen, mismo volumen.

Ideal para desarrollo y pruebas: una base de datos limpia en segundos.

En producción, las bases de datos pueden seguir donde están hoy.

02

Redes

Cómo se encuentran y se hablan los contenedores



Redes definidas por el usuario y DNS interno

Red bridge por defecto

web1 cliente

× wget http://web1 → bad address

Red red-curso (creada por nosotros)

api sql1 web1

✓ Server=sql1 · http://web1 (DNS interno)

En una red creada por vosotros, cada contenedor es accesible por su nombre. Nada de IP fijas, que cambian al recrear el contenedor.

Los contenedores de una misma red se ven sin publicar puertos. Publicar con -p es abrir al exterior: solo para lo que deba entrar desde fuera.

Volúmenes y redes

LAB

07

≈ 60 minutos

SQL Server con datos persistentes y contenedores que se encuentran por nombre.

- SQL Server sin volumen: comprobar que los datos se pierden
- Con un volumen con nombre: los datos persisten
- Bind mount: editar en vivo una web desde VS Code
- La red por defecto no resuelve nombres; una red propia sí
- Consultar SQL Server por nombre desde otro contenedor, sin publicar puertos

`labs/lab07-volumenes-y-redes/README.md`

03

Docker Compose

La aplicación completa en un único fichero



Lo que resuelve Compose

A mano

- docker network create …
- docker volume create …
- docker run … db (con 6 parámetros)
- Esperar a que SQL Server esté listo
- docker build … && docker run … api
- Y recordarlo todo para la próxima vez

→

Con Compose

- Un fichero compose.yaml, en Git junto al código
- docker compose up -d: lo crea todo, en orden
- docker compose down: lo elimina todo
- Mismo entorno para todo el equipo en un comando
- Declarativo: describes qué quieres, no los pasos

Anatomía de un compose.yaml

```
labs/lab08-docker-compose/compose.yaml

services:
  db:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
      ACCEPT_EULA: "Y"
      MSSQL_SA_PASSWORD: "${SA_PASSWORD}"
    volumes: ["sqldata:/var/opt/mssql"]
    networks: [backend]

  api:
    build: ./ConvocatoriasApi
    image: convocatorias-api:1.0.0
    environment:
      ConnectionStrings__Convocatorias: "Server=db;..."
    ports: ["8080:8080"]
    depends_on:
      db: { condition: service_healthy }
    networks: [backend, frontend]

volumes: { sqldata: {} }
networks: { backend: {}, frontend: {} }
```

services: cada contenedor de la aplicación.

image o build: imagen existente o carpeta con Dockerfile.

Server=db: el nombre del servicio es su nombre DNS.

${SA_PASSWORD}: se lee del fichero .env.

depends_on + healthcheck: la API espera a que la base de datos esté lista.

volumes y networks: se declaran al final y Compose los crea.

Los comandos de Compose que vais a usar

Comando	Qué hace
`docker compose up -d`	Crea y arranca todo lo que no existe o ha cambiado
`docker compose up -d --build`	Reconstruye las imágenes con build: y despliega
`docker compose ps`	Estado de los servicios, incluido el healthcheck
`docker compose logs -f api`	Logs de un servicio (o de todos, sin nombre)
`docker compose exec db bash`	Terminal dentro de un servicio
`docker compose up -d --scale api=3`	Varias instancias de un servicio
`docker compose down`	Elimina contenedores y redes; conserva los volúmenes
`docker compose down -v`	Elimina también los volúmenes: borra los datos
`docker compose config`	Muestra el fichero final con las variables sustituidas

Orden de arranque: depends_on y healthcheck

depends_on solo garantiza que un contenedor arranca antes que otro, no que esté listo para atender.

db arranca

Estado: health: starting. SQL Server todavía no acepta conexiones.

›

healthcheck

Cada 10 s Compose ejecuta sqlcmd ... -Q "SELECT 1" dentro del contenedor.

›

db healthy

El SELECT 1 responde. La base de datos está lista.

›

api arranca

condition: service_healthy se cumple: la API arranca y conecta a la primera.

Además, la aplicación debe tolerar que la base de datos tarde o se reinicie: reintentos en el arranque y en las conexiones.

Docker Compose

LAB

08

≈ 70 minutos

La API .NET con SQL Server y un nginx delante, en un único fichero.

- Completar el compose.yaml: API, variables y puertos
- Volumen y redes separadas para datos y tráfico
- Healthcheck y orden de arranque
- Comprobar la persistencia con down y up
- Ampliación: nginx como proxy inverso, escalar la API y actualizarla

`labs/lab08-docker-compose/README.md`

04

Buenas prácticas

Lo que separa una demo de algo que se puede llevar a producción



Buenas prácticas al construir imágenes

1 Multi-etapa y base mínima

Compilar con sdk, ejecutar con aspnet o chiseled. Menos tamaño, menos vulnerabilidades.

2 Versiones explícitas

Nunca latest en producción. Etiquetas con versión (1.4.2) y, en entornos muy controlados, el digest de la imagen.

3 Usuario sin privilegios

USER $APP_UID. Nada se ejecuta como root sin una razón muy concreta.

4 .dockerignore

Fuera bin, obj, .git y cualquier secreto. Contextos pequeños y limpios.

5 Orden pensado para la caché

De lo que menos cambia a lo que más: .csproj y restore antes que el código.

6 Escaneo de vulnerabilidades

docker scout, Trivy o el escáner del registro, en el pipeline, antes de publicar.

Buenas prácticas al ejecutar contenedores

1 Un proceso por contenedor

API, base de datos y proxy por separado. Se escalan, actualizan y diagnostican de forma independiente.

2 Configuración fuera

La misma imagen en todos los entornos. Variables de entorno y ficheros montados.

3 Secretos protegidos

Nunca en la imagen ni en Git. Ficheros .env protegidos, secrets de Compose o un gestor de secretos.

4 Datos en volúmenes

Contenedores desechables. Copias de seguridad de los volúmenes, no de los contenedores.

5 Logs a la salida estándar

Con rotación configurada, y recogidos por una plataforma central.

6 Salud, límites y exposición

Healthchecks, límites de CPU y memoria, restart policy y solo los puertos imprescindibles publicados.

05

De IIS a contenedores

Qué hace falta para desplegar en vuestros servidores



Qué cambia en la forma de desplegar

Hoy, con IIS	Con contenedores
Servidor Windows con IIS y el runtime de .NET instalado	Servidor con un motor de contenedores; el runtime va dentro de la imagen
Paquete Web Deploy o copia de ficheros	Imagen versionada publicada en un registro
web.config transformado por entorno	La misma imagen en todos los entornos, configuración inyectada
Sitio y application pool	Un contenedor por aplicación o servicio
IIS como servidor web y proxy (ARR)	Kestrel en el contenedor y un proxy inverso delante
Logs en ficheros del servidor	Logs a la salida estándar, enviados a una plataforma central
Marcha atrás: restaurar o redesplegar	Marcha atrás: arrancar la imagen anterior

Las piezas de la infraestructura



Feed NuGet interno

Compilar sin salir a nuget.org

Logs y monitorización

Plataforma central, Prometheus, Grafana

Secretos

Fuera de imágenes y repositorios

Copias de seguridad

Volúmenes y registro de imágenes

En un CPD sin salida a internet

Los servidores nunca descargan de registros públicos: todo pasa por el registro interno.

Imagen oficial

mcr.microsoft.com, Docker Hub. Se publica cada parche de .NET y del sistema base.

›

Zona con salida (DMZ)

Un registro que replica solo las imágenes aprobadas, o un equipo que hace docker save.

›

Escaneo y aprobación

Se revisan vulnerabilidades antes de admitir la imagen.

›

Registro interno

Imágenes base y vuestras aplicaciones. Fuente única para los hosts y el pipeline.

Lo mismo con NuGet: un feed interno (Nexus, Artifactory, Azure DevOps Artifacts) y un nuget.config que apunte a él.

¿Hace falta Kubernetes?

Lo visto en el curso. Dos hosts y el proxy repartiendo carga. El punto de partida recomendado.

Compose en cada servidor

Orquestador integrado en Docker, ficheros casi iguales a Compose. Varios servidores con poca complejidad añadida.

Docker Swarm

El estándar del sector (RKE2, k3s, OpenShift…). Muchas aplicaciones y equipos, autoescalado, despliegues sin corte. Requiere personal formado para operarlo.

Kubernetes

Complejidad y capacidad →

Una hoja de ruta posible

Fase 0 · Ya

Docker en los equipos de desarrollo. Bases de datos y dependencias de desarrollo con Compose. Sin impacto en producción.

›

Fase 1 · Piloto

Una aplicación interna, poco crítica, ya en .NET moderno. Registro, pipeline y dos hosts en preproducción.

›

Fase 2 · Producción

El piloto en producción detrás del proxy. Procedimientos de despliegue, marcha atrás, logs y parches.

›

Fase 3 · Adopción

Contenerizar cada aplicación al migrarla a .NET. Contenedores Windows solo donde se justifique.

›

Fase 4 · Orquestar

Con suficientes aplicaciones, valorar Swarm o Kubernetes.

Reto final

LAB

09

≈ 30 min en clase

Simular el camino de la aplicación hasta un servidor sin internet.

- Levantar un registro privado local (registry:3)
- Versionar la imagen y publicarla con docker push
- Desplegar con un compose de producción: sin build, sin puertos internos, con límites
- Imagen chiseled y plan B con save/load (trabajo autónomo)
- Repaso de la lista de buenas prácticas

`labs/lab09-reto-final/README.md`

Lo que os lleváis del curso

1 Fundamentos

Qué es un contenedor y por qué es desechable. Imágenes, capas y registros. Los comandos esenciales.

2 .NET en contenedores

Dockerfile multi-etapa, usuario sin privilegios, una imagen para todos los entornos. Y qué implica .NET Framework.

3 Aplicaciones completas

Volúmenes, redes y Docker Compose: la aplicación con su base de datos en un comando.

4 Criterio para decidir

Buenas prácticas, piezas de infraestructura, el escenario sin internet y una hoja de ruta por fases.

Próximos pasos y recursos

1 Practicar

Repetid los labs desde el repositorio y las grabaciones. Probad a contenerizar una aplicación vuestra ya migrada a .NET.

2 Empezar por desarrollo

La fase 0 no necesita a nadie más: SQL Server y dependencias en Compose para el equipo.

3 Documentación

docs.docker.com · learn.microsoft.com/dotnet/core/docker · github.com/dotnet/dotnet-docker · docs/DE-IIS-A-CONTENEDORES.md

GRACIAS

¿Preguntas?

Jorge López Blasco · linkedin.com/in/jorgelobl

